Firmware Block Implementation for the Proto-DUNEII Hit Finding Architecture with the HLS Tool

Joshua Horswill

January 4, 2021

Contents

1 DUNE's DAQ architecture; Envisaging the Alternatives

1.1 Overview

- There are four TPC (time projection chamber) modules of liquid argon; each module contains 150 anode plane assemblies (APA's) with 2560 wires per APA. The Front End LInk eXchange system contains the focus of this project.
- There are 10 fibres/data streams in the APA, each with 256 wires - hence the APA total of 2560
- The readout window per event is between 3-5.5 ms for a 2.25 ms drift time.
- The FELIX architecture consists of 10 MGT modules that feed data into 10 Hit Finders (HF), each of which produce hits by processing ADC values in parallel. This enters the HF's via the data router, and then travel into 10 sets of 4 TPG block processors. These are multiplexed (combined) by the arbitrator into the Central Router (CR) interface.

  What are these things?

- The MGT module processes the APA data stream into computational bits.
- The data router transforms the data from the MGT for processing in the TPG block, where the hit finding occurs.
- The data router receives multiple-channel, single clock-tick packets. The packets initially exist in a format containing the first sample for all channels, and when processed by the data router are converted into a format subsisting of all the samples for a given channel i.e. multiple clock ticks of data.
- Each TPG unit processes 64 wires, and so there are 4 of them ($4 \times 64 = 256$, the number of wires in a fiber).
- The TPG data consists of a header and payload. The header determines the source wire, the time stamp, and the origin of the data. The payload is 64 bits of ADC sample data for hit finding.
- There is an AXI4-stream protocol for unpacking and reordering of data in the TPG block.
- There are 4 general signals for various instructions in each component:

  1. tvalid denotes if the bus data is valid
  2. tuser flag denotes the end of a frame
  3. tlast denotes the end of packet

4. tready is used to apply back-pressure if the processing units ahead cannot process the data fast enough (all throughput is frozen).

- The TPG block strips the header in the Header Stripper/Combiner block (see figure 1) and sends it to the output of the block. Meanwhile the 64 ADC values are processed by the PedSub (pedestal subtraction), FIR (finite impulse response), and HF sub-blocks in the chain. The data router is then signalled to send the next packet.

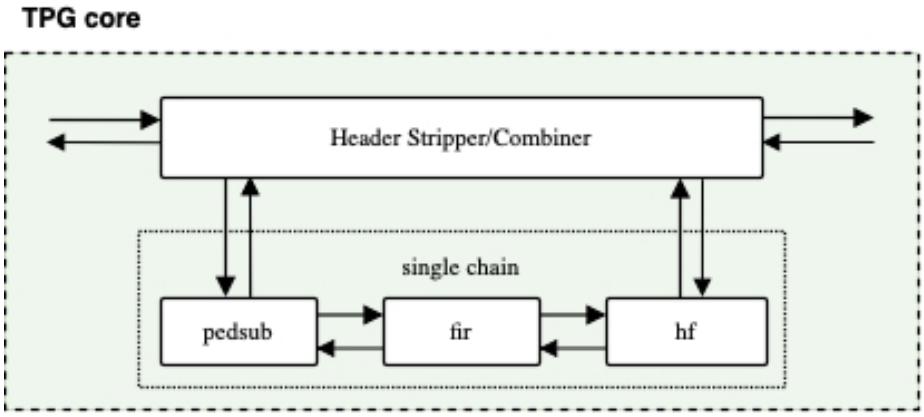

Figure 1: Focus on the header stripper/combiner and the sub-block chain [1].

See figure 2 for a visual representation of the HF structure and figure 3 for a similar view of the of the FELIX architecture.

1.2 Why liquid argon?

From the TPC wiki page.

1. Noble element: vanishing electronegativity (tendency of an atom to attract a pair of electrons that it shares with another atom) means that electrons produced by ionizing radiation will not be absorbed as they drift towards the detector.

2. Argon scintillates when an energetic charged particle passes by, releasing scintillation photons, the number of which is proportional to the energy deposited by the passing particle.

3. It's density is 1000 times denser than Nygren's design, which is important for neutrino physics as it increases the chance of a neutrino-nucleon interaction.

4. Argon is pretty cheap too.

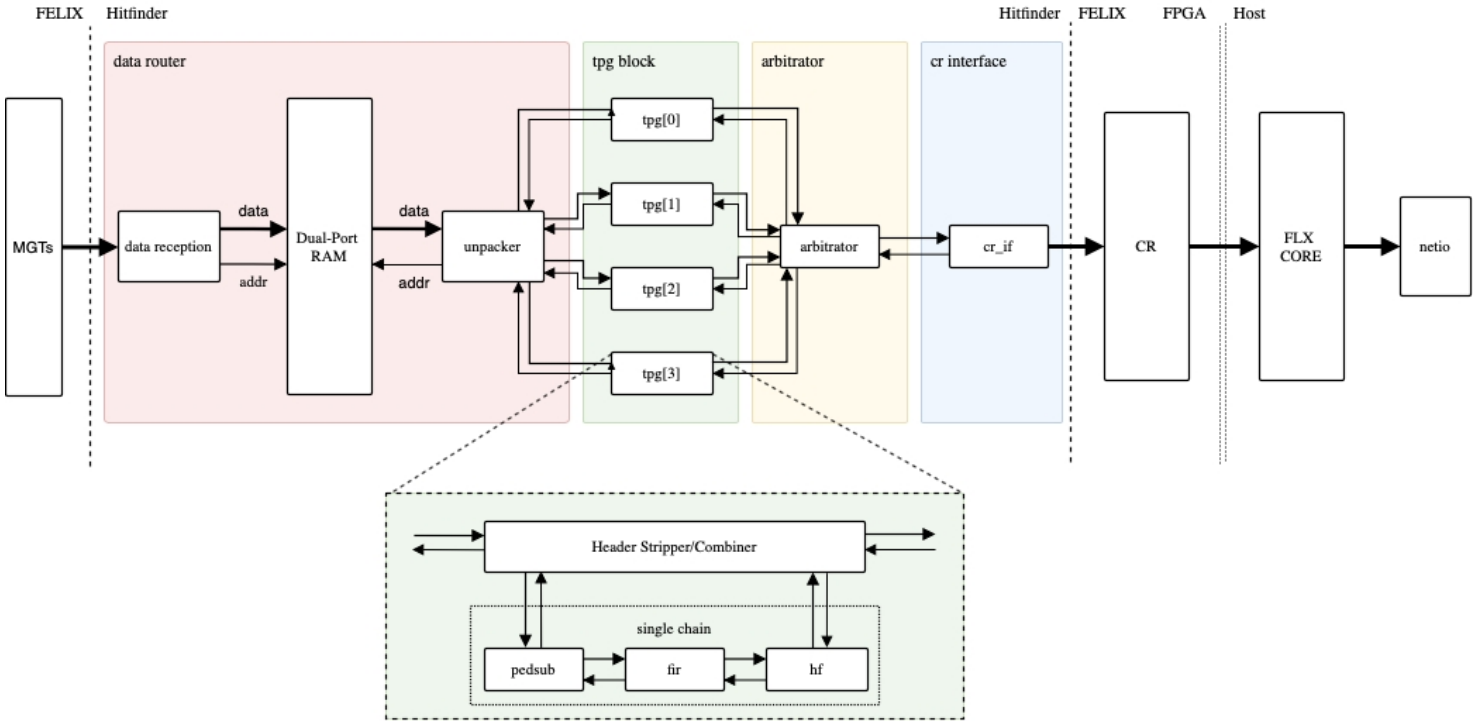

Figure 2: Hit Finder Firmware Architecture [1].

1.3 System Architecture

We have to consider some factors when studying the DUNE DAQ (data acquisition) system:

1. Hardware and maintenance cost per channel.
2. Technology life-cycle (TLC) of the components. We need long-lasting essential components such as the FPGA (Field Programmable Gate Array) and the network components.
3. Component power consumption: deep underground components with higher power consumption will increase the cooling and running cost.
4. DUNE will run for 20 years so the reliability and maintenance of components are more important than in other large scale projects.
5. Emphasis on modular design of hardware and firmware to avoid a change affecting the rest of the sub-modules.
6. Fast R&D and fast firmware modification: If the tools used are more abstract and are rooted in C++, this will appeal to the vast majority of the users of this tech (physicists).

Requirements for the architectural design of DUNE DAQ:

1. Enough bandwidth provided for the data stream
2. Supernova detection, trigger buffer
3. Detector performance, electronics' noise, level and source (?)

There are strong trade offs between the above requirements:

- High bandwidth networking is expensive, but could be avoided using trigger primitives (?), but the efficiency of this trigger depends on the detector noise performance.
- One can reduce the noise in the acquired signal by applying a digital filter in the front end DAQ before the trigger generating algorithm. However, this is at the cost of using lots of gates and resources in the FPGA.

DAQ architecture can be split into two parts:

- Front end DAQ, comprised of electronics that acquire the data stream from readout-electronics from the TPC detector, so it can process and deliver it to the back end (computation farm).
- Back-end DAQ, comprised of an offline storing, reconstruction computation farm.

2 FELIX

FELIX is a PCIe based high-throughput approach for interfacing front-end and trigger electronics in the ATLAS Upgrade framework:

- "The ATLAS Phase-I upgrade (2018) requires a Trigger and Data Acquisition (TDAQ) system able to trigger and record data from up to three times the nominal LHC instantaneous luminosity."
- The FELIX system provides the infrastructure to achieve this in a scalable, detector agnostic and easily upgradeable way.
- FELIX enables the reduction of custom electronics in favour of software running on commercial servers.
- FELIX is a PC-based networking gateway with the FPGA mounted on PCIe Gen3 cards (high speed connections used in motherboards to connect GPUs, SSD's, wifi chips e.t.c)

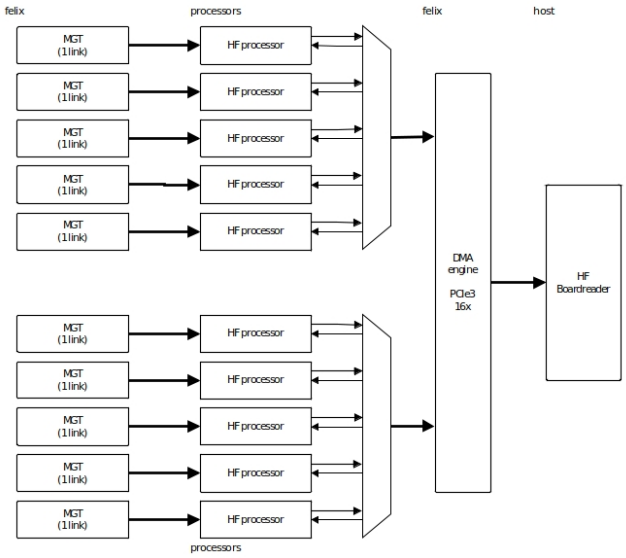

Figure 3: Overview of the Felix architecture in ProtoDUNE [1]

2.1 Data transfer

On one side of the FELIX system, the GigaBit Transceiver (GBT) architecture and a protocol developed by CERN provides 4.8 Gb/s radiation-hard optical link for data transmission from the on-detector front-end electronics. When all the data is multiplexed, the GBT protocol provides up to 42 independent

data links, sharing the same fibre. The GBT frame consists of a 116 bit data (payload) and a 4 bit header, the latter of which is used to align the frame at the receiver side. It can be DATA (0101) or IDLE (0110).

FELIX has to handle the Time, Trigger and Control (TTC) system [1], that is based on the recovered LHC clock, by forwarding the machine-synchronous trigger information. This info will be distributed to the aforementioned on-detector electronics over low and fixed latency GBT links. It will also be distributed to the first-level trigger systems.

For the FrontEnds which needs a higher throughput toward FELIX, a customized lightweight FULL mode is defined. The link speed is 9.6 Gb/s, but as data is encoded in 8b/10b a maximum user payload of 7.68 Gb/s can be achieved, which is an improvement on the standard 4.8 Gb/s [2].

3 4 Tonne Demonstrator for Large-Scale Dual-Phase Liquid Argon TPC's

10 kilo-tonne dual-phase liquid argon TPC is one of the main detector options considered for DUNE. The detector relies on amplification of the ionisation charge in ultra-pure argon vapour, which holds several advantages over the single-phase liquid argon TPC. LAr TPC's offer unprecedented 3D imaging capabilities and the functionality of a homogeneous calorimeter. High-resolution imaging is the key to efficient background noise rejection.

When a powerful neutrino beam is hypothetically shot from the United States' Fermilab to these LAr TPC's in the Sanford Underground Research facility in South Dakota, DUNE aims to search for the leptonic CP violation and determine the ordering of the neutrino masses.

Another objective is to extend the sensitivity of proton lifetime in a variety of possible decay channels, and collect high-statistics neutrino samples from atmospheric and astronomical sources.

3.1 Quick note on CP violation and neutrinos

CPV is well established in QCD, where the Cabibbo-Kobayashi-Mashawa (CKM) matrix is complex. Theoretically, this could arise from complex Yukawa couplings[2] and/or a relative CP-violating phase in the vacuum expectation value (VEV) of the Higgs fields.

You would expect an entirely analagous mechanism to arise in the lepton sector of the standard model (SM), leading to leptonic CPV (LCPV). The discovery of neutrino oscillation provides evidence for non-vanishing neutrino masses and leptonic mixing.

[1] a system that executes one or more sets of tasks according to a pre-determined and set task schedule

[2] Interaction between a scalar field and Dirac field, could be used to describe nuclear force between nucleons, but mainly used to describe the coupling between the Higgs field and the massless quark/lepton fields

We are not sure why neutrino masses are so small. We think there is a connection between low-energy neutrino physics and leptogenesis, which could provide insight into the scenario that led to the baryon asymmetry of the universe.

3.2 Detector details

Single phase liquid argon detectors, readout by wires immersed in the liquid, do not require the amplification of the ionized electron charge. The charge is localised using alternative induction and collection wire planes with different orientations. This was tested by ProtonDune-SP at CERN.

The dual phase version is between 20 and 50 ktonnes and, in contrast to the previous example, relies on the amplification of ionisation charges in the ultra-pure cold argon vapour layer above the liquid, obtaining low-energy detection thresholds with high signal-to-noise ratio over drift distances exceeding 10m.

"Ionisation charges produced by neutrino interactions would drift vertically towards the liquid-vapour boundary where they are extracted into the gas phase, amplified by Large Electron Multipliers (LEMs), and collected on finely segmented anodes"; the electron extraction, amplification and collection are performed inside a 3 x 1 m^2 frame called the charge readout plane (CRP). Bear in mind the detector prototype volume is 3 x 1 x 1 m^3.

Five photo-multiplier tubes are mounted underneath the TPC drift cage. They detect the Argon (prompt S1) scintillation photons formed from passing charged particles, as well as the (proportional S2) secondary scintillation light from the electroluminescence of the electrons extracted in the argon vapour. S1 is at least several orders of magnitude higher in detection amplitude than S2 in a tested TPC.

A wavelength shifter converts the deep ultraviolet photons to the visible spectrum within the sensitivity of the receiving PMT photo-cathode. These photo-cathodes give the absolute time on an event T_0. The CRP applies an electric field in the liquid above 2 kV/cm to move the electrons into the vapour.

There was a measured correlation between the electroluminescence in the gas from S2 scintillation, and the collected charge from the CRP system, as you would intuitively expect [3].

4 HLS

High-level synthesis (HLS) is an automated design process that interprets an algorithmic description of a desired behavior and creates digital hardware that implements that behavior.

Synthesis begins with a high-level specification of the problem, where behavior is generally decoupled from low-level circuit mechanics such as clock-level timing. Early HLS explored a variety of input specification languages, although recent research and commercial applications generally accept synthesizable subsets of ANSI C/C++/SystemC/MATLAB.

The code is analysed, architecturally constrained, and scheduled to transcompile into a register-transfer level (RTL) design in a hardware description language (HDL), which is in turn commonly synthesized to the gate level by the use of a logic synthesis tool.

RTL is a design abstraction which models a synchronous[3] digital circuit in terms of the flow of data between hardware registers[4], and the logical operations performed on those signals.

RTL is used in HDLs i.e. languages that describe the structure and behaviour of electronic circuits, and more commonly digital logic circuits. It allows for circuitry simulation and digital analysis. HDLs look like programming languages such as C, but the main difference is that HDLs include the notion of time. RTLs are used in HDLs to create high-level representations of a circuit, from which lower-level representations and ultimately actual wiring can be derived. Design at the RTL levle is typical practise in modern digital design.

RTL clock cycle data is given in Vivado performance estimate data, as it can read how long a certain block, loop or function in the code took to complete; this is called the latency.

4.1 HLS Design Flow

1. Compile, execute (simulate), and debug the C algorithm.

   - This involves validating that the C program correctly implements the required processes.

   There is also post-synthesis validation that compares the synthesised RTL and C programs to see if they perform the same actions.

   In order for the former to occur, there must be a test bench C script that includes a top-level function main() as well as the function to be synthesised.

2. Synthesize the C algorithm into an RTL implementation, optionally using user optimization directives.

3. Generate comprehensive reports and analyze the design.

   After C synthesis, output files are stored in a directory named `syn` which contain folders for each RTL output format, and a report folder. The report folder contains files reporting on the performance and design corresponding to the top-level function and each individual subfunction.

4. Verify the RTL implementation using a pushbutton flow.

5. Package the RTL implementation into a selection of IP formats.

[3] A circuit that changes the state of memory elements in a way synchronised by a clock signal

[4] Circuits composed of flip flops - a sub circuit that has two stable states used to store state information (a basic storage element in sequential logic). These hardware registers have the ability to read or write multiple bits at a time and can use an address to select a particular register. They are used as an interface between the software and the computer peripherals.

4.1.1 Inputs

- C function written in C, C++, or SystemC

  This is the primary input to Vivado HLS.

- Constraints

  Constraints are required and include the clock period, clock uncertainty, and FPGA target. The clock uncertainty defaults to 12.5% of the clock period if not specified.

- Directives

  Directives are optional and direct the synthesis process to implement a specific behavior or optimization.

- C test bench and associated files

  Vivado HLS uses the C test bench to simulate the C function prior to synthesis and to verify the RTL output using C/RTL co-simulation.

4.1.2 Outputs

- RTL implementation files in hardware description language (HDL) formats

  This is the primary output from Vivado HLS. Using Vivado synthesis, you can synthesize the RTL into a gate-level implementation and an FPGA bitstream file.

  Vivado HLS packages the implementation files as an IP block for use with other tools in the Xilinx design flow. Using logic synthesis, you can synthesize the packaged IP into an FPGA bitstream.

- Report files

  This output is the result of synthesis, C/RTL co-simulation, and IP packaging.

4.2 Optimization

Different optimization directives can be applied before C synthesis:

- Instruct a code task to execute in a pipeline, allowing the next execution of a task to begin before the current one finishes
- Specify a latency for the completion of functions and loops (one of the reasons for this is to match the FPGA clock frequency).
- Specify a limit to the resources used
- Select the I/O protocol to make sure the final design can be connected to other hardware blocks with the same I/O protocol.

4.3 Analysis of the C Synthesis Results

The first part of the analysis process involves the content of the synthesis reports. We can analyse the performance estimates which involve:

- Timing - the target clock frequency, clock uncertainty and the fastest achievable clock frequency of the design.
- Latency - The latency and initiation interval (II) for the block and any sub-blocks instantiated in the main block. Recall the latency is the number of clock cycles it takes for the program to produce the output, and the initiation interval is the number of clock cycles before new inputs can be processed. A summary of this is included, with greater detail provided in the subsequent section.
- Without pipeline directives (see the section on optimization) the latency is one cycle less than the II.
- Utilization estimates - resources such as the LUTs, FF's, DSP48s used to implement the synthesised design. This also includes the BRAM used, and whether it is single-port or dual-port.

The analysis perspective in Vivado HLS provides a view of the module hierarchy - the resources and latency for each block in the RTL code. This can be observed in more detail in the performance profile for a selected block.

Within the analysis perspective is the schedule viewer tab, which allows for the identification of loop dependencies preventing parallelism, timing violations and data dependencies. One can filter the operations by type or by cluster.

4.4 HLS Directives

Directives applied to an object or scope will apply it to all the sub-objects within. For example, applying a directive to an interface applies it to the top-level function of the C program, as this is the scope containing the interface. Applying a directive to a loop applies it to all objects contained within the loop.

Directives can be stored as a Tcl command within directives.tcl, or within the source code as a pragma. Do the former for transient directives and the latter for likely permanent directives[5], for all solutions. Pragmas must be validated during C synthesis and will output errors if configured incorrectly.

`#pragma` commands for directives include:

- `#pragma HLS PIPELINE II=1` to get Vivado to try to reduce the initiation interval to 1 clock cycle by initiating process during other processes.
- `#pragma HLS unroll [factor = N]` to try to run N iterations of the loop in parallel (omitting the factor = N automatically runs all iterations in parallel) which results in an increase in resources used, but a higher system performance and more throughput.

[5]Such as TRIPCOUNT and INTERFACE

- `#pragma HLS DATAFLOW` to enable task-level pipelining, allowing functions and loops to overlap in their operation, increasing throughput.
- `#pragma HLS allocation instances=<list>` (nextline) `limit=<value> <type>` specifies the instance restrictions to limit resource allocation in the implemented kernel.

with many more available at `https://www.xilinx.com/html_docs/xilinx2019_1/sdaccel_doc/hls-pragmas-okr1504034364623.html#zof1504034359187`.

4.5 Arbitrary Precision Types

The different numerical data types are as follows:

- char - 8 bit
- short - 16 bit
- int - 32 bit
- long long - 64 bit
- float - 32 bit
- double - 64 bit
- int16_t and int32_t - 16 and 32 bit exact width integer types

Creating hardware means that accurate bit-widths are required. Using standard C data types for hardware design results in unnecessary hardware costs. Operations can use more LUTs and registers than needed for the required accuracy, causing delays that may exceed a given clock cycle, increasing the overall latency. Smaller bit widths (e.g using 17 bit data types for 17 bit multiplication rather than the standard C 32 bit data type) result in hardware operators which are smaller and faster. More logic can then fit in the FPGA and can operate at a higher clock frequency.

To implement arbitrary ap_[u]int precision types in C++, include header file ap_int.h in the project source file. Change the bit types to ap_int⟨N⟩ or ap_uint⟨N⟩, where N is a bit-size ranging from 1 to 1024. You can set this limit higher by `#define AP_INT_MAX_W N`, but setting this too high can cause slow software compile and run times.

4.6 Interface Synthesis

Vivado HLS creates three types of ports on the RTL design:

1. Clock and Reset ports: ap_clk, ap_rst

2. Block level interface protocols: ap_start, ap_done, ap_ready and ap_idle.

   These are the signals that control the block independently of any port-level I/O protocols. ap_start dictates when the block can start processing data, ap_ready dictates whether it can accept new inputs, and you can guess what ap_idle and ap_done are responsible for.

3. Port level interface protocols: created for each argument in the top-level function and the function return.

   The I/O protocol depends on the type of C argument and on the switch default option. Port level IO protocols sequence the data in and out of the block once operation has been started by the above protocols. If there is no I/O protocol associated with the block output port, it is difficult to know when to read the data. Therefore it is always a good idea to use one on an output.

The `DATA_PACK` optimisation directive can pack all the elements of a struct into a single wide vector, allowing all the members of the struct to be read and written to simultaneously. This overrides the default process for structs, where they are decomposed into their member elements and ports are implemented separately for each element. More data can be consequently accessed in a given clock cycle.

When this is the case, Vivado HLS can automatically unroll the loops consuming this data, improving the throughput. This can be controlled by the `config_unroll` command and the `tripcount_threshold` option. The syntax is `config_unroll -tripcount_threshold <N>` where N is the minimum number of loops where the loop is not automatically unrolled.

If a struct contains arrays, those arrays can be optimised using the `ARRAY_PARTITION` directive to partition the array or the `ARRAY_RESHAPE` directive to partition the array and also re-combine the partitioned elements into a wider array. This can greatly improve performance. You cannot use `DATA_PACK` and the the array directives in tandem; they are mutually exclusive.

5 Pedestal Subtraction Algorithm

The pedestal subtraction algorithm is the first sub-block in the TPG core, fed 64 sample words from the Header-Stripper/Combiner, shown in figure 1. The first 6 words (16 bit data types) are stripped from the incoming samples and sent to the `pedsub` block, where the 'background noise' of the ADC values is removed. The algorithm is implemented as follows:

- The algorithm works by scanning each sample within the packet, subtracting the pedestal from each ADC value and adjusting the pedestal and accumulator variables according to the size of each ADC value. The firmware runs a AXI4 interface between blocks , which operates through a series of boolean signals mentioned in section 1.1. `tkeep` and `treset`

are signals that are not mentioned previously, the former of which goes high when the incoming data should be processed, and the latter goes high when the entire process must be reset and restarted. The algorithm only runs when `tready` is low and `tvalid` and `tkeep` are high. `tlast` and `tuser` go high at the end of a packet. `treset` goes high when an error in the FELIX interface occurs.

- The user inputs an estimate of the median background value to subtract from the first few samples, as a temporary buffer before the pedestal value is changed by the algorithm.
- The algorithm also starts with an associated accumulator variable, initially equal to zero, that directly changes the pedestal value once it reaches ± 10. How does this accumulator variable change?
- The accumulator changes by ± 1 depending on the boolean (`ADCvalue` > `pedestal`). If this boolean is true, the accumulator increases by one, and false means a decrease by one.
- The next condition is: if before a scan the accumulator is $+10$, the pedestal increases by one, and the accumulator is reset to zero. Then the next ADC value is subtracted by this new increased pedestal. The opposite is the case for an accumulator of -10; the pedestal decreases by one and the accumulator resets.
- At the end of each packet, the current value for the channel is stored and then retrieved from a state save/restore mechanism, for the next packet entering the channel.

Eventually, with a large enough flux of packets, one would obtain a pedestal value appropriate for that input channel, giving more useful ADC values.

5.1 Pedestal Subtraction Testbench

The testbench of any design is one of the most important parts of any HLS project, and usually requires more time to complete than the source algorithm. The main purpose of the testbench is to validate the design when editing the top function so that it can C-simulate within the HLS compiler, synthesize and RTL/cosimulate to then be exported into an IP block. See section 4.1 for more detail on the firmware design flow.

The first testbench design generated an array of random ADC integer values within a reasonable range, and found the ideal pedestal value for that range. Subsequently, the testbench function scanned those ADC values with the subtraction algorithm and validated that the pedestal value converged to the expected value, or was equal to that value after a certain number of packets. This was acceptable to test the first iteration but it did not validate the adjusted ADC values output from the algorithm.

The output from a VHDL implementation of the `pedsub` algorithm was used to compare the output data of the HLS algorithm from one wave of packets travelling through 64 channels. This included a reading of the AXI4 signals, and a contingency for random raises of treset and tready to see if the data was affected. This testbench also included the state save/restore mechanism to see if the output would be affected by multiple packet waves entering a given channel. The result was as expected, which meant the design was ready for IP format packaging.

5.2 Internalising the State Save/Restore (SSR) Mechanism into the Top Function

Once the algorithm VHDL design was verified by the Vivado HLS software, it had to be developed within the Vivado project manager, where the implementation can be tested within Questasim (more commonly known as modelsim) to check all signals are being received and output as expected. Unfortunately this simulation revealed that too much of the state save restore

References

[1] Konstantinos Manolopoulos. Processing framework & hit finder algorithm, Jul 2020.

[2] J Anderson, K Bauer, A Borga, H Boterenbrood, H Chen, K Chen, G Drake, M Dönszelmann, D Francis, D Guest, et al. Felix: a pcie based high-throughput approach for interfacing front-end and trigger electronics in the atlas upgrade framework. *Journal of Instrumentation*, 11(12):C12023, 2016.

[3] Benjamin Aimard, Ch Alt, J Asaadi, M Auger, V Aushev, D Autiero, MM Badoi, A Balaceanu, G Balik, L Balleyguier, et al. A 4 tonne demonstrator for large-scale dual-phase liquid argon time projection chambers. *Journal of Instrumentation*, 13(11):P11003, 2018.